

Contents

□ Smart Samarpan Academy — Complete Interview Preparation Guide	2
	2
PART 1 — PROJECT INTRODUCTION	2
	2
1. What is this project?	2
2. What real-world problem does it solve?	2
3. What was the motivation behind building it?	2
4. Who are the target users?	2
5. What is the basic working idea?	3
□ 30-Second Explanation (Interview)	3
□ 1-Minute Interview Explanation	3
	4
PART 2 — IMPACT	4
	4
1. Who benefits from this project?	4
2. What problem does it solve?	4
3. What measurable impact can I honestly claim?	4
4. What metrics are relevant?	5
	5
PART 3 — FEATURES	5
	5
Complete Feature Matrix	5
Feature Priority Ranking	6
	7
PART 4 — TECH STACK	7
	7
Frontend Technologies	7
Backend Technologies	10
	13
PART 5 — COMPLETE ARCHITECTURE	13
	13
Visual Architecture	13
Architecture Explanation (Simple English + Hinglish)	15

□ Smart Samarpan Academy — Complete Interview Preparation Guide

Your Project: A full-stack AI-powered mathematics learning platform

Built With: React + Vite + Tailwind CSS | Node.js + Express | MongoDB | Google Gemini AI | Razorpay

PART 1 — PROJECT INTRODUCTION

1. What is this project?

Samarpan Math Academy is a full-stack web application — an **online mathematics learning platform** specifically designed for students in **Classes 9 to 12**. It lets students browse courses, watch video lectures, take AI-generated quizzes, generate mathematical formulas using AI, track their learning progress, and make payments for premium courses. It also has a dedicated **admin dashboard** where administrators can manage courses, lectures, and users.

Simple meaning (Hinglish): Yeh ek online math padhai ki website hai — jisme student courses buy kar sakta hai, video lectures dekh sakta hai, AI se quiz de sakta hai, aur admin courses manage kar sakta hai.

2. What real-world problem does it solve?

Mathematics is a subject where students need **practice, instant feedback, and personalized help**. Traditional coaching classes: - Are expensive and limited by geography - Don't provide instant quiz generation tailored to a student's weak area - Don't track progress lecture-by-lecture - Can't generate formulas on demand

This platform solves these problems by bringing **AI-powered personalization** to math education.

3. What was the motivation behind building it?

The motivation was to create an **accessible, affordable, and intelligent** math learning platform that: - Uses **AI (Google Gemini)** to dynamically create quizzes and formulas — so content is never stale - Allows students to learn at their own pace with **video lectures** - Provides a **progress tracking** system so students know what they've completed - Has a **payment system** (Razorpay) for monetization

4. Who are the target users?

User Type	Description
Students	Class 9-12 students who want to learn mathematics online
Teachers/Admins	Educators who create and manage course content
Parents	Who want to buy courses for their children

5. What is the basic working idea?

Student visits website

- Browses courses (free to see)
- Registers / Logs in (Email or Google OAuth)
- Buys a course (Razorpay payment)
- Watches video lectures
- Marks lectures as completed (progress tracking)
- Takes AI-generated quizzes (Gemini AI)
- Generates math formulas using AI
- Views performance history

Admin logs in

- Creates/Edits/Deletes courses
- Adds/Edits/Deletes video lectures
- Manages users (change roles, delete)
- Views platform statistics

□ 30-Second Explanation (Interview)

“I built **Samarpan Math Academy**, a full-stack online learning platform for Class 9-12 math students. Students can browse and purchase courses, watch video lectures, and track their progress. The unique part is the **AI integration** — I used **Google Gemini API** to dynamically generate MCQ quizzes and mathematical formulas based on any topic the student types. The backend is built with **Node.js, Express, and MongoDB**, and the frontend uses **React with Vite**. I also implemented **Razorpay payment gateway** and **Google OAuth** for authentication.”

□ 1-Minute Interview Explanation

“So I built this project called **Samarpan Math Academy**. The idea was simple — I wanted to create a platform where math students, specifically Classes 9 through 12, can learn at their own pace using video lectures and AI-powered tools.

The platform has **two types of users**: students and admins. Students can register using email-OTP verification or Google OAuth, browse available courses, make payments through Razorpay, and then access video lectures. What makes it different is the **AI features** — I integrated Google Gemini API so students can generate custom quizzes on any math topic with adjustable difficulty, and also generate mathematical formulas with LaTeX rendering.

On the admin side, there's a complete dashboard for managing courses, lectures, and users with role-based access control.

The tech stack is **MERN** — React with Vite and Tailwind on the frontend, Node.js with Express on the backend, and MongoDB for the database. I also handled things like **JWT authentication**, **bcrypt password hashing**, **email OTP verification using Nodemailer**, and **file uploads using Multer** for course images and lecture videos.

The frontend is deployed on **Vercel** and the backend on **Render**.”

PART 2 — IMPACT

1. Who benefits from this project?

Beneficiary	How
Math students (Class 9-12)	Get structured courses, video lectures, AI quizzes, formula help
Teachers / Content creators	Can upload and manage courses via admin dashboard
Self-learners	Can practice with AI-generated quizzes at any difficulty

2. What problem does it solve?

- **No personalized practice:** AI generates quizzes based on specific topic + difficulty → personalized
- **No instant formula reference:** AI formula generator gives formulas with LaTeX rendering instantly
- **No progress tracking:** Students can see exactly which lectures they've completed
- **Payment:** Enables course monetization via Razorpay

3. What measurable impact can I honestly claim?

[!IMPORTANT] I cannot confirm actual user counts, revenue, or performance metrics from the code. Below are **structural facts** from the codebase:

Metric	What I can confirm from code
Course categories	4 categories supported (Class 9, 10, 11, 12)
Mongoose models	6 database collections (User, Courses, Lecture, Payment, Progress, QuizResult)
API endpoints	31 REST API endpoints
AI integrations	2 (Quiz Generation + Formula Generation via Gemini)

Metric	What I can confirm from code
Auth methods	3 (Email/Password, OTP Verification, Google OAuth)
Payment gateway	Razorpay integration with order creation + signature verification
Quiz result fields tracked	8 per quiz (topic, difficulty, total, correct, wrong, score, percentage, answers)
Admin features	Full CRUD for courses, lectures, users + stats dashboard

4. What metrics are relevant?

If you were to deploy and track, relevant metrics would be: - Number of registered users - Number of courses created - Number of quiz attempts (stored in QuizResult collection) - Average quiz score/percentage - Number of payments processed - Course completion rate (from Progress collection) - Number of formulas generated - API response time for AI features

PART 3 — FEATURES

Complete Feature Matrix

#	Feature	Purpose	Why Needed	How It Works
1	User Registration + OTP Verification	Secure account creation	Prevents fake accounts, verifies email	Register → OTP sent via Nodemailer → Verify OTP → Account created
2	Email/Password Login	Standard authentication	Most users expect this	Email + Password → bcrypt compare → JWT token issued (7 days)
3	Google OAuth Login	One-click sign in	Reduces friction, faster onboarding	Redirect to Google → Callback → Create/Find user → JWT issued
4	Forgot/Reset Password	Password recovery	Users forget passwords	Email → OTP sent → Verify OTP → New password hashed + saved
5	Course Browsing	View available courses	Students need to discover content	GET all courses → Display cards with image, price, duration, category
6	Course Purchase (Razorpay)	Monetization	Platform needs revenue	Create Razorpay order → Open checkout → Verify signature → Enroll user
7	Video Lectures	Core learning content	Primary way students learn	Video files served from /uploads, played via HTML <video>

#	Feature	Purpose	Why Needed	How It Works
8	Progress Tracking	Track completion	Students need to know what's done	Mark lecture complete → Store in Progress collection → Show checkmarks
9	AI Quiz Generation (Gemini)	Practice & assessment	Personalized practice at any level	Select topic + difficulty → Gemini generates MCQs → Student answers → Score
10	Quiz Result History	Track performance over time	Students want to see improvement	Save results to DB → Display history table with score, date, topic
11	AI Formula Generator	Quick formula reference	Students need formulas on demand	Type topic → Gemini returns formulas → Rendered in LaTeX via KaTeX
12	Admin Dashboard	Platform management	Admin needs oversight	Stats (users, courses, lectures counts) displayed on dashboard
13	Admin Course CRUD	Manage courses	Content needs to be created/updated	Create/Read/Update/Delete courses with image upload
14	Admin Lecture CRUD	Manage lectures	Videos need to be added/removed	Create/Read/Update/Delete lectures with video upload
15	Admin User Management	Manage users	Need to control access, remove bad actors	List users, change roles, delete users
16	Role-Based Access Control (RBAC)	Security	Different users need different permissions	isAuth + isAdmin middleware checks role
17	Responsive UI	Mobile-friendly	Students use phones	Tailwind CSS responsive classes + mobile hamburger menu

Feature Priority Ranking

☐ HIGH PRIORITY (You MUST understand deeply)

Feature	Why High Priority
AI Quiz Generation	Most unique, most asked about, involves external API, complex logic
Authentication System (Register + OTP + Login + Google OAuth)	Core security, multiple flows, JWT, bcrypt, Nodemailer
Razorpay Payment	Real-world payment handling, signature verification, security-critical
RBAC (Role-Based Access Control)	Middleware pattern, security architecture
Course + Lecture CRUD (Admin)	Full CRUD operations, file uploads, database operations

☐ MEDIUM PRIORITY

Feature	Why Medium
Progress Tracking	Good DB design question, shows state management
AI Formula Generator	Similar to quiz but simpler flow
Quiz Result History	Shows data persistence and retrieval
Admin Dashboard (Stats)	Demonstrates aggregation queries

□ LOW PRIORITY

Feature	Why Low
Course Browsing	Simple GET + display
Responsive UI	Standard Tailwind usage
About Page	Static content
Footer/Header	Standard components

PART 4 — TECH STACK

Frontend Technologies

1. React.js (v18.2)

TERM: React

→ **Simple meaning:** A JavaScript library for building user interfaces using reusable components.

→ **Why used in my project:** To build the interactive frontend — course pages, quiz UI, admin dashboard.

→ **Example:** Every page (Home, Login, QuizPage) is a React component.

Aspect	Detail
What?	JavaScript library for building UIs with a component-based architecture
Why?	Component reuse, virtual DOM for performance, huge ecosystem, React hooks for state
Where?	Entire frontend — all 14 pages + 4 components + 6 admin pages
How it communicates?	Makes HTTP requests to Express backend via Axios
Alternatives	Angular, Vue.js, Svelte
Why React is better here?	Largest community, easiest to find help, Context API sufficient for this project size
Limitations	Not a full framework (no built-in routing, state management) — needed react-router-dom and Context API separately

Aspect	Detail
Interview Q	"Why React over Angular?" → React is a library not a framework, more flexible, component-based, Virtual DOM

2. Vite (v5.2)

TERM: Vite

- **Simple meaning:** A fast build tool and dev server for modern web projects.
- **Why used:** Replaces Create React App — much faster hot reload and builds.
- **Example:** `vite.config.js` configures the project, `npm run dev` starts Vite dev server.

Aspect	Detail
What?	Next-generation frontend build tool using native ES modules
Why?	Extremely fast dev server start and Hot Module Replacement (HMR)
Where?	vite.config.js , build process
Alternatives	Webpack, Create React App (CRA), Parcel
Why Vite?	CRA is deprecated, Webpack is slow. Vite uses esbuild for pre-bundling → 10-100x faster
Limitations	Some older plugins may not be compatible

3. Tailwind CSS (v4.1)

TERM: Tailwind CSS

- **Simple meaning:** A utility-first CSS framework where you style using class names directly in HTML.
- **Why used:** Fast styling without writing custom CSS files for each component.
- **Example:** `className="bg-gray-900 text-white p-4 rounded-lg"` applies dark background, white text, padding, rounded corners.

Aspect	Detail
What?	Utility-first CSS framework
Why?	Rapid UI development, consistent design, responsive utilities, dark mode
Where?	Every component uses Tailwind classes. tailwind.config.js
Alternatives	Bootstrap, Material UI, Chakra UI, vanilla CSS
Why Tailwind?	More customizable than Bootstrap, no pre-built components to override, smaller bundle (purges unused CSS)
Limitations	Long class names in JSX, learning curve for utility classes

4. Axios (v1.7)

TERM: Axios

- **Simple meaning:** A library for making HTTP requests (like GET, POST) from the browser.
- **Why used:** To send API requests from React frontend to Express backend.
- **Example:** `axios.post("/api/user/login", { email, password })`.

Aspect	Detail
What?	Promise-based HTTP client for browser and Node.js
Why?	Cleaner API than native <code>fetch</code> , automatic JSON parsing, interceptors, error handling
Where?	UserContext.jsx , CourseContext.jsx , all pages making API calls
Alternatives	Native <code>fetch</code> API, SWR, React Query
Why Axios?	Better error handling, supports request/response interceptors, works in older browsers

5. React Router DOM (v6.23)

TERM: React Router

→ **Simple meaning:** A library that lets you navigate between different pages in a React app without full page reload.

→ **Why used:** For client-side routing — `/login`, `/courses`, `/admin/dashboard`, etc.

Aspect	Detail
Where?	App.jsx defines all routes
Alternatives	Next.js (file-based routing), TanStack Router

6. Recharts (v3.1)

TERM: Recharts

→ **Simple meaning:** A charting library for React to create graphs and charts.

→ **Why used:** To display quiz results as a Pie Chart on the quiz results page.

Aspect	Detail
Where?	QuizPage.jsx — PieChart showing correct vs wrong answers
Alternatives	Chart.js, D3.js, Victory
Why Recharts?	Built specifically for React, declarative API, easy to use

7. KaTeX (v0.16)

TERM: KaTeX

→ **Simple meaning:** A library for rendering mathematical formulas beautifully in the browser (like textbook equations).

→ **Why used:** Quiz questions and formulas contain math — KaTeX renders LaTeX notation.

Aspect	Detail
Where?	QuizPage.jsx , FormulaGenerator.jsx
Alternatives	MathJax
Why KaTeX?	Faster rendering than MathJax, lighter weight

8. React Google OAuth (@react-oauth/google v0.12)

TERM: Google OAuth

→ **Simple meaning:** Allows users to log in using their Google account instead of creating a new password.

→ **Why used:** Reduces signup friction — one-click login.

Aspect	Detail
Where?	main.jsx (GoogleOAuthProvider), Login.jsx

9. react-hot-toast (v2.4)

TERM: Toast Notifications

→ **Simple meaning:** Small popup messages that appear briefly to show success/error messages.

→ **Why used:** User feedback for actions like “Login Successful”, “Course Created”, etc.

Backend Technologies

10. Node.js

TERM: Node.js

→ **Simple meaning:** A JavaScript runtime that lets you run JavaScript outside the browser — on a server.

→ **Why used:** So the entire app (frontend + backend) uses one language: JavaScript.

Aspect	Detail
What? Why?	JavaScript runtime built on Chrome's V8 engine Full-stack JavaScript, non-blocking I/O, npm ecosystem
Alternatives Why Node?	Python (Django/Flask), Java (Spring Boot), Go Same language as React frontend, async by default, fast for I/O operations
Limitations	Single-threaded (not ideal for CPU-heavy tasks), callback complexity

11. Express.js (v5.1)

TERM: Express

→ **Simple meaning:** A minimal web framework for Node.js that helps you create API routes easily.

→ **Why used:** To define all the backend API endpoints.

Aspect	Detail
Where?	index.js , all route files
Alternatives	Fastify, Koa, NestJS, Hapi
Why Express?	Simplest, most popular, huge middleware ecosystem

Aspect	Detail
Limitations	Minimal by design — no built-in ORM, validation, etc.

12. MongoDB + Mongoose (v8.16)

TERM: MongoDB

→ **Simple meaning:** A NoSQL database that stores data as JSON-like documents (not tables like MySQL).

→ **Why used:** Flexible schema, natural fit with JavaScript/JSON.

→ **Example:** User document: { name: "Suman", email: "...", role: "user", subscription: [...] }

TERM: Mongoose

→ **Simple meaning:** A library that provides structure (schemas) to MongoDB documents and makes database operations easier.

→ **Why used:** Defines schemas for User, Course, Lecture, Payment, Progress, QuizResult.

Aspect	Detail
Where?	database/db.js , all files in models/
Alternatives	PostgreSQL, MySQL, Firebase Firestore
Why MongoDB?	Schema-less flexibility, stores JSON natively, great with Node.js
Limitations	No joins (use <code>populate</code>), no ACID transactions by default, eventual consistency

13. JSON Web Token (JWT v9.0)

TERM: JWT

→ **Simple meaning:** A secure token (like a digital ID card) given to a user after login, used to prove identity on each request.

→ **Why used:** Stateless authentication — server doesn't need to store sessions.

Aspect	Detail
Where?	controllers/user.js (token generation), middlewares/isAuth.js (token verification)
Alternatives	Session-based auth (express-session), OAuth tokens, Passport.js
Why JWT?	Stateless (scalable), works across domains, contains user info

14. bcrypt (v6.0)

TERM: bcrypt

→ **Simple meaning:** A library that hashes (scrambles) passwords so they're stored securely — can't be read even if database is stolen.

→ **Why used:** Never store plain text passwords.

Aspect	Detail
Where?	controllers/user.js — <code>bcrypt.hash(password, 10)</code> and <code>bcrypt.compare()</code>
Alternatives	Argon2, scrypt

15. Multer (v2.0)

TERM: Multer

- **Simple meaning:** A middleware for handling file uploads in Express.
- **Why used:** Course images and lecture videos need to be uploaded.

Aspect	Detail
Where?	middlewares/multer.js — configures disk storage to uploads/
Alternatives	Busboy, Formidable, direct cloud upload (S3)

16. Nodemailer (v7.0)

TERM: Nodemailer

- **Simple meaning:** A library to send emails from Node.js.
- **Why used:** Sends OTP emails for account verification and password reset.

Aspect	Detail
Where?	middlewares/sendMail.js — Gmail SMTP, port 465, SSL
Alternatives	SendGrid, AWS SES, Mailgun

17. Razorpay (v2.9)

TERM: Razorpay

- **Simple meaning:** An Indian payment gateway that lets you accept online payments.
- **Why used:** Students pay for courses via Razorpay checkout.

Aspect	Detail
Where?	index.js — order creation + signature verification
Alternatives	Stripe, PayU, Cashfree
Why Razorpay?	Best for Indian market, supports UPI/cards/netbanking, good docs

18. Google Gemini API

TERM: Gemini API

- **Simple meaning:** Google's AI model API — you send it text and it generates intelligent responses.
- **Why used:** Two features — AI quiz generation and AI formula generation.

Aspect	Detail
Where?	index.js — /api/course/quiz/generate and /api/formula/generate
Model used	gemini-2.0-flash
Alternatives	OpenAI GPT, Claude, Hugging Face
Why Gemini?	Free tier available, good for structured output, fast

19. Google Auth Library (v10.2)

TERM: OAuth2Client

- **Simple meaning:** Library for handling Google sign-in on the server side.
- **Why used:** Implements Google OAuth callback flow.

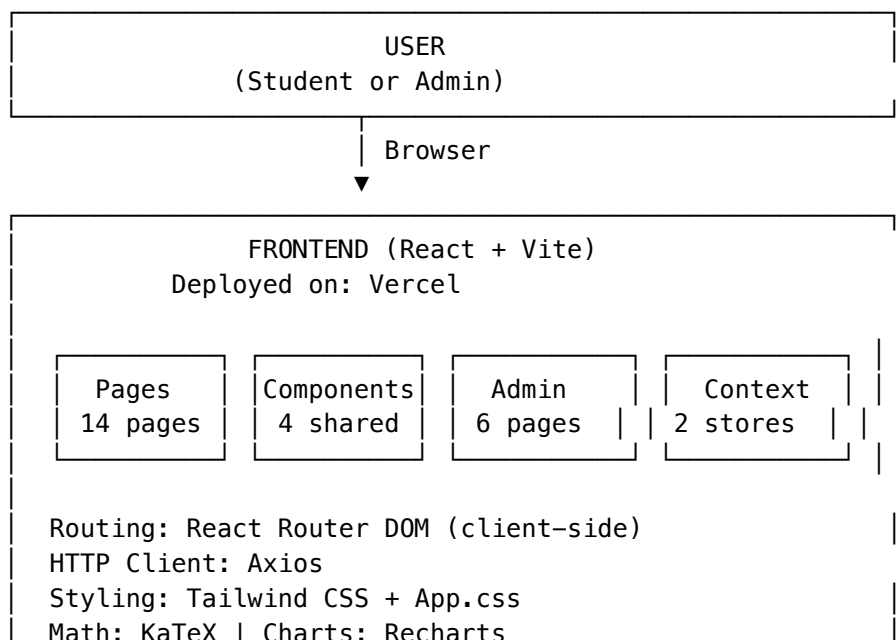
20. CORS (v2.8)

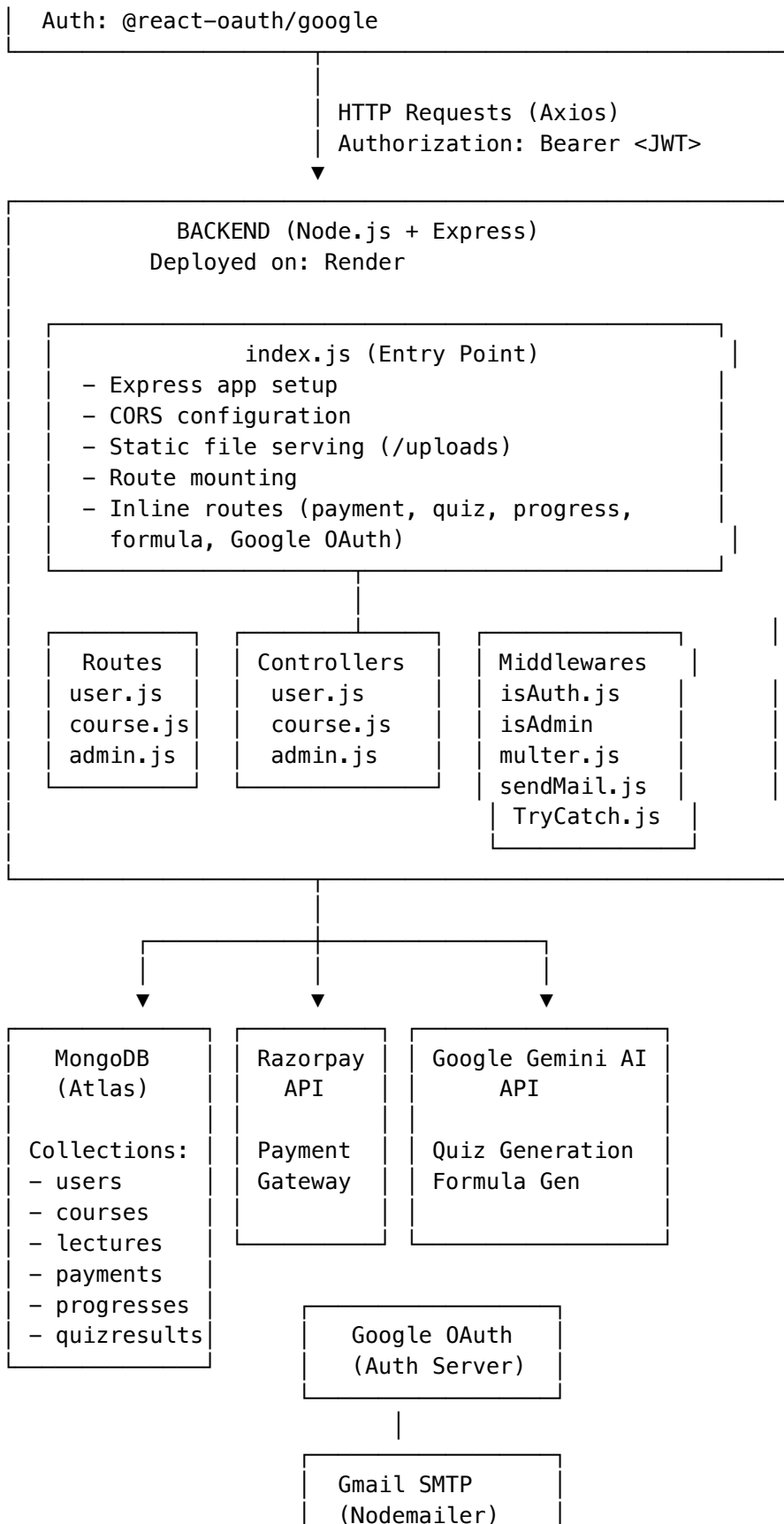
TERM: CORS

- **Simple meaning:** Cross-Origin Resource Sharing — allows your frontend (different URL) to talk to your backend (different URL).
- **Why used:** Frontend on Vercel (samarpan-math-academy.vercel.app) needs to access backend on Render.

PART 5 — COMPLETE ARCHITECTURE

Visual Architecture





Architecture Explanation (Simple English + Hinglish)

1. User (Student ya Admin)

User browser mein website open karta hai. Yeh frontend se interact karta hai.

2. Frontend (React + Vite + Tailwind)

Yeh sab kuch jo user dekhta hai — pages, forms, buttons, videos. React components se bana hai. Vite fast development deta hai. Tailwind se styling hoti hai.

Important: Frontend apne paas koi data store nahi karta permanently. Sab kuch backend se maangta hai via API calls (Axios).

3. API Request (Axios + JWT)

Jab user kuch karta hai (login, course buy, quiz start), frontend ek HTTP request bhejta hai backend ko. Agar user logged in hai, toh request mein JWT token header mein jaata hai: Authorization: Bearer eyJhbGciOi...

4. Backend (Express + Node.js)

Server request receive karta hai. Pehle **middleware** check hota hai: - **isAuth**: JWT token valid hai? User kaun hai? - **isAdmin**: Kya user admin hai? - **multer**: Kya file upload ho rahi hai?

Phir **controller** function chalta hai jo actual logic handle karta hai.

5. Database (MongoDB)

Controller database se data fetch karta hai ya save karta hai Mongoose ke through. 6 collections hain — Users, Courses, Lectures, Payments, Progress, QuizResults.

6. External APIs

- **Google Gemini**: Quiz aur formula generate karne ke liye
- **Razorpay**: Payment process karne ke liye
- **Google OAuth**: Google se login ke liye
- **Gmail SMTP**: OTP emails bhejne ke liye

7. Response

Backend response bhejta hai (JSON format mein) → Frontend receive karta hai → UI update hoti hai → User ko result dikhta hai.